Konzeptionsdokument – IU Studien-Dashboard

Modul: Objektorientierte und funktionale Programmierung mit Python (DLBDSOOFPP01_D)

Phase: 1 – Konzeptionsphase

Name: Sascha Schumbera

Matrikelnummer: IU14153477

Datum: März 2026

Idee und Vorgehen

Seit November 2025 studiere ich Angewandte Künstliche Intelligenz an der IU – berufsbegleitend, 20 Stunden pro Woche neben dem Job. Ohne feste Semestergrenzen verliert man schnell den Überblick, ob man noch im Plan liegt. Ich möchte deshalb ein persönliches Studien-Dashboard entwickeln, das die wichtigsten Kennzahlen auf einen Blick zeigt.

Der Bedarf ist direkt aus meiner Situation abgeleitet. Python mit tkinter ist gut dokumentiert, braucht keine externen Pakete und lässt sich sinnvoll in OOP-Konzepte übersetzen. Ein Spreadsheet würde technisch dasselbe leisten – der Mehrwert liegt in der Lernkomponente: objektorientierte Modellierung und GUI-Programmierung gezielt zu üben.

Vorgehen: Ziele und UML-Modell definieren, technische Machbarkeit mit Testprogrammen absichern, dann Implementierung in Phase 2.

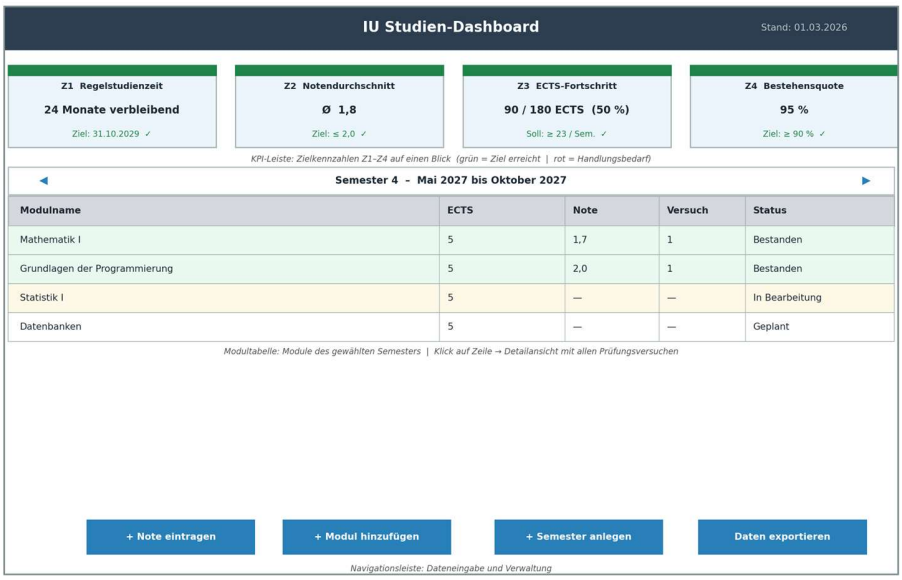
1. Ziele

Das Dashboard überwacht vier Kennzahlen:

1. **Regelstudienzeit einhalten (Ziel: 31.10.2029):** Prognostizierter Abschlusstermin auf Basis des aktuellen ECTS-Lerntempos vs. Zieldatum 31.10.2029 (48 Monate, 8 Semester).
2. **Notendurchschnitt ≤ 2,0:** ECTS-gewichteter Durchschnitt aller abgeschlossenen Module, mit grüner/roter Markierung je nach Zielerreichung.
3. **ECTS-Soll pro Semester (≥ 23 ECTS):** 180 ECTS / 8 Semester = 22,5, aufgerundet auf 23 als Sollwert. Anzeige als Fortschrittsbalken je Semester.
4. **Erstversuchs-Bestehensquote ≥ 90 %:** Anteil bestandener Erstversuche an allen Prüfungen. Wiederholungen kosten Zeit, die ich neben dem Job kaum habe.

2. Dashboard-Skizze

Das Dashboard gliedert sich in drei Bereiche. Die Skizze zeigt einen exemplarischen Zustand mitten im Studium – alle vier Zielkennzahlen sind auf einen Blick sichtbar:



KPI-Leiste (oben): Vier Kacheln für Z1–Z4, farbig (grün = Ziel erreicht, rot = Handlungsbedarf).

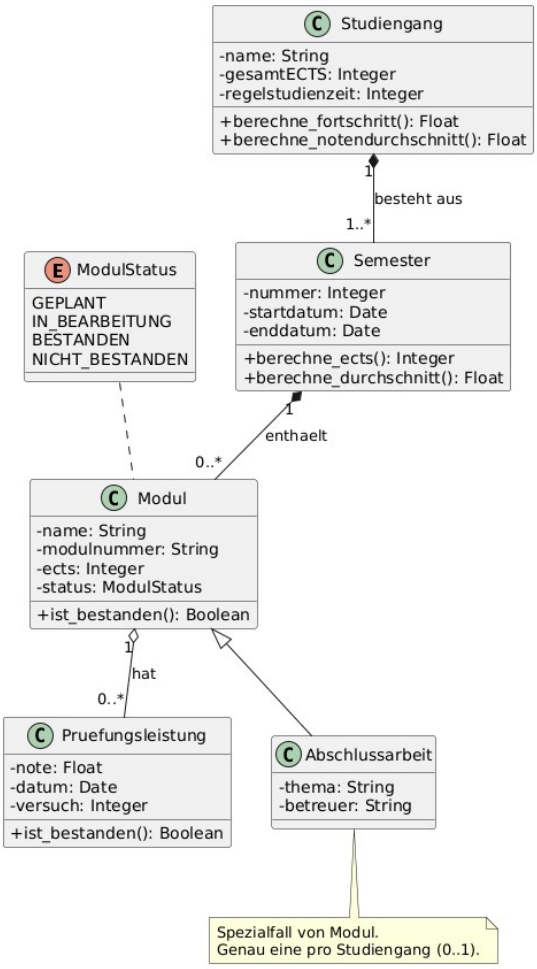
Modultabelle (Mitte): Module des Semesters mit ECTS, Note, Versuch und Status. Klick auf Zeile öffnet Detailansicht mit allen Prüfungsversuchen.

Navigationsleiste (unten): Buttons zur Dateneingabe und Verwaltung.

3. UML-Klassendiagramm

3.1 Entity-Klassendiagramm

Das Diagramm wurde mit PlantUML erstellt und ist bewusst sprachunabhängig gehalten (String, Integer, Float statt Python-Typen).



3.2 Diskussion der Modellierungsentscheidungen

Komposition Studiengang → Semester und Semester → Modul: Semester existieren nur im Kontext eines Studiengangs – wird der Studiengang entfernt, werden auch alle Semester hinfällig. Gleiches gilt für Module innerhalb eines Semesters. Komposition ist hier die passende Beziehungsart, da die Lebensdauer vollständig am übergeordneten Objekt hängt.

Aggregation Modul → Prüfungsleistung: Hier war ich mir anfangs nicht sicher. Eine Prüfungsleistung (Note, Datum, Versuch) ist eigentlich ein eigenständiger Datensatz – sie könnte theoretisch auch ohne das Modul-Objekt sinnvoll sein, z. B. als Prüfungsnachweis. Deshalb Aggregation. Komposition wäre genauso vertretbar, wenn man die Bindung enger sieht.

Vererbung – Abschlussarbeit erbt von Modul: Die Bachelorarbeit hat ECTS, Status und Note wie jedes Modul, braucht aber zusätzlich Thema und Betreuer. Abschlussarbeit erbt von Modul und ergänzt nur das Spezifische. Wichtig: Abschlussarbeit ist ein Spezialfall von Modul und existiert genau einmal im gesamten Studiengang – nicht je Semester. Das UML-Diagramm zeigt die Vererbungsbeziehung; die Einschränkung auf genau eine Instanz ist im Modell ergänzend dokumentiert (Kardinalität 0..1 bezogen auf den Studiengang).

Abstraktion: Modul fasst alle Prüfungsformate zusammen (Klausur, Portfolio, Projekt). Das Attribut status wurde auf Basis des Tutor-Feedbacks als Enumeration modelliert (ModulStatus: GEPLANT, IN_BEARBEITUNG, BESTANDEN, NICHT_BESTANDEN), da ein Aufzählungstyp die erlaubten Zustände klar begrenzt und Fehleingaben verhindert. Abschlussarbeit ist die einzige Spezialisierung, die eigene Attribute benötigt.

Getter und Setter im UML-Klassendiagramm: Auf Basis des Tutor-Feedbacks wurden einfache Getter- und Setter-Methoden aus dem UML-Diagramm entfernt, da sie Implementierungsdetails darstellen und nicht zur fachlichen Modellierung gehören. In der späteren Python-Umsetzung werden diese über den @property-Dekorator realisiert.

Zuordnung von get_note(): Gemäß dem Information-Expert-Prinzip ist die Methode get_note() fachlich der Klasse Prüfungsleistung zuzuordnen, da dort die Note gespeichert ist. Eine Platzierung in Modul würde zu unnötiger Kopplung führen. Im überarbeiteten Diagramm (v2) wurde get_note() daher aus Modul entfernt.

4. Machbarkeitsüberprüfung

Entwicklungsumgebung: Python 3.14 (aktuelle Stable-Version) + VS Code 1.112.0 mit Python-Extension. VS Code habe ich gewählt, weil es kostenlos ist, eine gute Python-Integration mit Autovervollständigung und integriertem Debugger mitbringt und ich damit bereits vertraut bin. venv sorgt für isoliertes Paketmanagement.

Bibliotheken: Alle drei sind Teil der Standardbibliothek – keine externen Pakete nötig. tkinter für die grafische Oberfläche (ausreichend für ein einfaches Dashboard, keine Installation nötig), json für die Datenpersistenz (menschenlesbar, direkt les- und schreibbar), datetime für die Datumsberechnungen (Restlaufzeit als KPI).

Quellcodeverwaltung: Git 2.53 mit lokalem Repository. Damit habe ich jederzeit eine Versionshistorie, kann Änderungen rückgängig machen und den Entwicklungsfortschritt nachvollziehen – auch für ein Einzelprojekt sinnvoll.

Datenspeicherung: JSON-Datei. Für ein Single-User-Dashboard ausreichend, kein Datenbankserver nötig, menschenlesbar und mit dem eingebauten json-Modul vollständig les- und schreibbar. Eine Umstellung auf SQLite wäre später möglich, wenn die Datenmenge wächst.

4.1 Test 1 – Grafische Oberfläche (tkinter)

Frage: Kommandozeile oder GUI? Ich habe mich für tkinter entschieden – ein Dashboard lebt von der schnellen Lesbarkeit, das funktioniert mit GUI besser als mit reinem Textoutput. Der Test zeigt Eingabefeld, Button und farbige KPI-Anzeige in Aktion:

```
import tkinter as tk

def zeige_fortschritt():
    ects_ist = int(ects_eingabe.get())
    prozent = ects_ist / 180 * 100
    farbe = "green" if prozent >= 50 else "red"
    ergebnis_label.config(
```

```
        text=f"Fortschritt: {ects_ist} / 180 ECTS ({prozent:.1f} %)",
        fg=farbe)

fenster = tk.Tk()
fenster.title("IU Studien-Dashboard – KPI-Test")
ects_eingabe = tk.Entry(fenster); ects_eingabe.pack()
tk.Button(fenster, text="Berechnen", command=zeige_fortschritt).pack()
ergebnis_label = tk.Label(fenster, text=""); ergebnis_label.pack()
fenster.mainloop()
```

Ergebnis: Fenster öffnet sich, ECTS-Eingabe wird verarbeitet, Fortschritt wird farbig angezeigt (grün ab 50 %, sonst rot). Funktioniert.

4.2 Test 2 – Datenpersistenz (json)

Der Test speichert eine vollständige Semesterstruktur mit Modulen und Noten in eine JSON-Datei, liest sie wieder ein und berechnet daraus den gewichteten Notendurchschnitt:

```
import json, os
daten = {"studiengang": "B.Sc. Angewandte Kuenstliche Intelligenz",
         "semester": [{"nummer": 1, "module": [
            {"name": "Grundlagen der Programmierung", "ects": 5, "note": 1.7},
            {"name": "Mathematik I", "ects": 5, "note": 2.0}]}]}
with open("studium_data.json", "w", encoding="utf-8") as f:
    json.dump(daten, f, ensure_ascii=False, indent=2)
with open("studium_data.json", "r", encoding="utf-8") as f:
    geladen = json.load(f)
module = geladen["semester"][0]["module"]
gesamt_ects = sum(m["ects"] for m in module)
durchschnitt = sum(m["note"] * m["ects"] for m in module) / gesamt_ects
print(f"Erworbene ECTS: {gesamt_ects}")
print(f"Gewichteter Notendurchschnitt: {durchschnitt:.2f}")
```

Ausgabe: Erworbene ECTS: 10 | Gewichteter Notendurchschnitt: 1.85 | JSON-Persistenz: erfolgreich.

4.3 Test 3 – Datumsberechnung (datetime)

Die Restlaufzeit bis Studienende ist einer der vier KPIs. Der Test prüft, ob sich das mit der datetime-Standardbibliothek berechnen lässt:

```
from datetime import date

start = date(2025, 11, 1)
ziel_ende = date(2029, 10, 31)
heute = date.today()
verbleibend_tage = (ziel_ende - heute).days

print(f"Studienbeginn:  {start.strftime('%d.%m.%Y')}")
print(f"Ziel-Abschluss: {ziel_ende.strftime('%d.%m.%Y')}")
print(f"Verbleibend:    {verbleibend_tage} Tage ({verbleibend_tage // 30} Monate)")
print("Datumsberechnung: erfolgreich.")
```

Ausgabe (zum Zeitpunkt des Tests): Studienbeginn: 01.11.2025 | Ziel-Abschluss: 31.10.2029 | Verbleibend: 1322 Tage (44 Monate) | Datumsberechnung: erfolgreich.